ABESEC Ghaziabad

Department of Computer Science & Engineering

SYNOPSIS REPORT

(Session 2026-27)

Project Title: Guardrail: Irreversibility-Aware Action Gating for Human-in-the-Loop Control of Autonomous AI Agents				
Project Type (application, product, research, review etc.)			Research + Product (applied research leading to a deployable software product)	
	Name	**Roll Number**	**Section**	**Signature**
Group member (1)				
Group member (2)				
Group member (3)				
Project Guide				
Signature		**Remarks:**		
Date of submission				

1. Problem Introduction

1.1 Motivation

Agents built on large language models have moved from producing text to performing actions: they issue refunds, send mail, modify databases, and edit and deploy source code on a user's behalf. An incorrect sentence can be ignored, whereas an incorrect action changes the state of the world and may not be recoverable. This authority is being granted to systems whose reliability remains modest, the best agent in the original WebArena study having completed 14.41 percent of tasks against a human baseline of 78.24 percent, and errors compound across long workflows, so a non-trivial per-step error rate is a permanent design assumption. The failure mode is documented: in July 2025 a commercial coding agent deleted a live production database during a declared code freeze and then falsely reported that rollback was impossible. The instructive detail is that the safeguard existed only in the prompt and was never enforced in the execution path. Every widely deployed gate decides when to consult a human from model confidence, applied as a single global threshold. Confidence estimates how likely the agent is to be wrong and says nothing about what happens if it is: an action the model is 80 percent sure of gets the same verdict whether it renames a temporary file or authorises an irreversible transfer. No one-dimensional threshold resolves this, because confidence and consequence are orthogonal quantities. Static allow and deny lists fail differently, being context free while reversibility is contextual: sending a message is benign in general and irrecoverable when the recipient is a regulator. The 2026 IBM and Ponemon study finds that 92 percent of organisations suffering an AI-related breach lacked adequate access controls.

1.2 Project Objective

The objective is to design, implement and empirically validate a gating layer that decides when an autonomous agent must pause and consult a human on the basis of the predicted cost of undoing the proposed action rather than its confidence in it, and to show that it dominates confidence gating on avoiding irreversible errors while minimising interruptions. Writing U(a, s, h) for undo cost, P_err for the probability that the action is wrong with respect to user intent, and C_int for the cost of interrupting the operator, the layer escalates when P_err x U exceeds C_int, subject to a bounded interruption budget. The specific objectives are:

- Construct an irreversibility taxonomy over three domains (file system, source control and deployment, financial and transactional), decomposing undo cost into recovery latency, monetary exposure and social exposure.
- Build a rollback-instrumented sandbox that executes an action, attempts an automated undo and records whether the prior state was in fact restored, and release a labelled dataset of at least 10,000 actions from it.
- Train an undo-cost regressor under an asymmetric quantile loss with conformal calibration, so that the policy consumes a distribution-free upper bound, and a detector for contextual and compositional irreversibility.
- Implement an expected-value escalation policy with learned per-class thresholds, a bounded interruption rate and batching, trained against realised decision cost.
- Package the system as a Model Context Protocol proxy and Python SDK, and evaluate it against six baselines on tau-bench and SWE-bench with gating overhead measured alongside safety and autonomy.

1.3 Scope of the Project

Three domains are covered, chosen because they span the range of undo cost and because automated rollback verification is feasible for each: file system, source control and deployment, and financial or transactional operations against mock services with synthetic records. Two environments are used, tau-bench for genuine database writes and SWE-bench for source-control trajectories; communication and network domains are future extensions. The layer gates before execution only, intercepting at the function-call boundary shared by mainstream frameworks and at the Model Context Protocol transport, and analyses one agent executing one task. Improving agent capability, adversarial defence against deliberate prompt injection, legal determination and hardened production deployment are outside scope. The project assumes tool calls can be intercepted, that sandbox undo cost approximates production cost within an action class, and that the reviewer is reachable, expensive and not perfectly accurate, so reviewer accuracy is an evaluation parameter rather than an assumed oracle. Training uses a single GPU, and no real customer or personal data enters the pipeline.

2. Related Previous Work

Agent capability benchmarks such as WebArena, tau-bench and SWE-bench supply the evaluation substrate here, offering realistic environments and write-capable tools, but all score task success uniformly: an agent that fails by giving up and one that fails by deleting the user's data both receive zero, so severity goes unmeasured. Agent safety evaluation is closer but categorical: ToolEmu elicits risky behaviour in an emulated sandbox, R-Judge benchmarks risk awareness and AgentDojo evaluates prompt injection, while runtime monitors and GuardAgent issue permit-or-block verdicts over action types. That work answers whether an action is risky, usually as a coarse label and often after execution, and never produces a continuous cost estimate. Its nearest relative is AI control work on trusted monitoring, in which a weaker trusted model audits a stronger untrusted one under an auditing budget, but audits on suspicion of misbehaviour rather than on measured cost of reversal.

Reversibility itself is well developed in reinforcement learning, where side-effect penalties, attainable-utility preservation and self-supervised reversibility estimation all presuppose a defined state space, a transition model and a resettable simulator. An agent calling a payments interface has none of them: reachability is not computable, the environment cannot be reset, and undo cost is a property of the external world. The inheritance is conceptual, not mechanical. Uncertainty alignment is the closest framing and the one we argue is incomplete: KnowNo applies conformal prediction so a planner asks for help when its prediction set is ambiguous, and a calibrated error probability is necessary for our rule but not sufficient, because the same probability warrants opposite decisions depending on what is at stake. Two older literatures are overlooked: compensating transactions and sagas in distributed databases, which give the vocabulary for exact inverses, compensating actions and partial restoration on which our rollback axis is built; and work on interruption cost, showing that it depends on timing and falls when interruptions are batched. Decision-focused learning justifies training the policy against realised decision cost, but has not been applied to agent supervision under a budget. The gap is therefore specific: no published system predicts a continuous, empirically measured undo cost for an open tool call before it executes, and none converts such an estimate into a budgeted escalation decision.

3. Software and Hardware Requirements

Software: Ubuntu 22.04 LTS or Windows 11 with WSL2 and Python 3.11; PyTorch, scikit-learn, XGBoost and LightGBM for the regressor and detector heads; Hugging Face Transformers and sentence-transformers for encoding actions and state; MAPIE for conformal prediction; LangChain, AutoGen and the Model Context Protocol Python SDK as integration targets; tau-bench and SWE-bench as evaluation environments; Docker with Git worktrees and database snapshots for the sandbox; FastAPI, PostgreSQL and React for the proxy, audit log and console; NumPy, pandas, SciPy and statsmodels for analysis; MLflow, DVC, Git, pytest and GitHub Actions for engineering hygiene.

Hardware: No dedicated hardware investment is required. Three workstations of Core i5 or Ryzen 5 class with 16 GB RAM cover development, sandbox execution and console work; training uses one T4 or P100 class GPU through Google Colab, Kaggle or the institutional cluster. The sandbox host is a Linux virtual machine with 4 vCPU, 8 GB RAM and snapshot-capable storage, with about 100 GB reserved for trajectories, snapshots and logs. Hosted model interfaces serve rollouts within free-tier and academic credit.

4. Proposed Method

Guardrail is a layer interposed between an agent and its tools. Every proposed tool call is intercepted before execution and passed through three components: a regressor predicting how costly the action would be to undo, a detector for contextual and compositional irreversibility invisible from the action alone, and a policy engine comparing expected damage against the cost of interrupting a person, returning execute or escalate.

Step 1. Irreversibility taxonomy: actions are classified on four axes: domain; rollback mechanism (exact inverse, compensating action, partial restoration leaving residue, or none), following the compensating-transaction vocabulary; externality (contained, crossing an organisational boundary, or reaching a third party); and latency sensitivity, the rate at which reversibility decays. It is released as a specification with a JSON schema, validated by inter-annotator agreement on 200 actions.

Step 2. Sandbox and automated labelling: the sandbox snapshots the relevant state (file-system hash tree, database dump, mock ledger, outbound queue), executes the action, attempts the rollback mechanism indicated by the taxonomy under a bounded time budget, and verifies restoration by residual difference against the snapshot. Recovery latency is measured directly and monetary exposure read from the ledger. Social exposure cannot be measured this way and is assigned from the externality axis; this is a modelling assumption, is reported as a limitation, and every headline result is reported over a grid of component weights rather than at one setting. External events that destroy reversibility, such as a message being read or a commit pulled, are simulated by documented stochastic processes, not fixed rules.

Step 3. Undo-cost regressor and error probability: inputs are structural features of the call (tool name, argument types, destructive flags, wildcard scope), a sentence-encoder embedding that generalises to unseen tool names, and state features such as target sensitivity and whether the target is production. Because under-prediction permits a destructive action while over-prediction merely spends budget, training minimises an asymmetric quantile loss with quantile above 0.5; several heads give an interval and split conformal prediction converts it into a distribution-free bound, of which the policy consumes the upper end. P_err combines self-consistency across sampled rollouts with a calibrated classifier. Because sampling multiplies agent cost several times over, the gate is staged: a cheap feature screen resolves calls that are clearly safe or clearly catastrophic, and sampling runs only where the interval straddles the threshold. Added latency and token overhead are measured, not assumed away.

Step 4. Silent-irreversibility detector: a contextual head predicts an amplification factor on the regressor output from trajectory history and observed external events. A compositional head maintains a typed resource graph of which recovery affordances (primary, backup, snapshot, audit copy) remain available and scores each action for the affordances it would consume, so an action removing the last recovery path is amplified even where its intrinsic cost is low. Training data comes from a closed set of composition patterns verified at sequence level in the sandbox, extending the Step 2 loop without new machinery.

Step 5. Escalation policy engine: the engine learns a threshold per action class by minimising realised total cost, namely damage from unescalated errors plus interruption cost. Escalations are capped as a rate per hundred actions rather than a count per task, so long tasks are not starved; the effective threshold rises as the budget depletes; and non-blocking escalations in close succession are batched into one review. Reviewer accuracy below one is an evaluation parameter, so the reported ceiling reflects imperfect human review. Every escalation, verdict and triggering feature is written to an append-only audit log.

Step 6. Integration surfaces: the Model Context Protocol proxy is the primary artefact, sitting between the agent and its tool servers and requiring only an endpoint change. A Python SDK exposing a decorator and context manager wraps any tool function in about three lines and, because interception happens at the shared function-call boundary, one implementation covers all mainstream frameworks. A minimal console provides the review inbox, risk timeline and audit log.

Step 7. Evaluation: the system is compared against six reference policies.

B1 No gating	Executes every action. Upper bound on irreversible errors, lower bound on interruptions.
B2 Fixed-confidence gating	Escalate below one global confidence threshold. Dominant practice and primary comparison.
B2+ Per-domain confidence gating	Same rule, one threshold fitted per domain, so B2 is not disadvantaged by lower capacity.
B3 Entropy or self-consistency	Escalate on high predictive entropy or low agreement across sampled rollouts.
B4 Static allow and deny list	Escalate on membership of a hand-written danger list. Current industrial guardrails.
B5 Zero-shot LLM judge	A frontier model rates how costly the call would be to undo; the rating is thresholded.
B6 Oracle with perfect hindsight	Escalates exactly on actions that were in fact errors. The unachievable frontier limit.

All policies are swept across their threshold range under an equal tuning budget. The irreversible-error rate is the proportion of tasks containing at least one erroneous executed action that left a state which could not be restored; correct actions that are intentionally irreversible are excluded, and an action counts as erroneous when it diverges from the benchmark reference solution or violates its policy checks, fixed per environment before experiments begin. The remaining metrics are interruptions per task, realised damage cost, task success rate as a control against safety by refusal, gate overhead as median and 99th percentile latency with the token cost multiplier, and regression quality.

The headline result is a Pareto frontier relating irreversible-error rate to interruption frequency, and the claim under test is dominance: that consequence-aware gating reaches a region of that plane which B2 and B2+ cannot reach at any threshold. Ablations remove, in turn, the detector, the conformal calibration, the learned thresholds and the budget; the primary ablation fixes P_err at a constant so that gating on consequence alone is isolated from the quality of confidence estimation, and if that variant already dominates B2 the claim holds independently of how error probability is estimated. Generalisation is tested on a held-out domain, with significance from paired bootstrap and Wilcoxon tests over five seeds. Three risks dominate: sandbox cost may diverge from real cost, mitigated by validating a stratified subsample against expert judgement; high-cost actions are rare, mitigated by deliberate synthesis and cost-weighted sampling; and the scope is large, mitigated by drawing the phase boundary so that Semester VII alone yields a self-contained publishable contribution.

5. Deliverables

- D1. Irreversibility taxonomy over three domains and four axes: specification plus JSON schema.
- D2. Rollback-instrumented sandbox and verifier, containerised, with a test suite.
- D3. Open labelled undo-cost dataset of at least 10,000 actions, with protocol and datasheet.
- D4. Undo-cost regressor with conformal intervals: weights, inference code and benchmark report.

- D5. Silent-irreversibility detector with a sequence-level evaluation suite.
- D6. Expected-value policy engine with learned thresholds, bounded interruption rate and batching.
- D7. Protocol proxy and gating SDK as an installable package, plus console and audit log.
- D8. A workshop or conference paper per semester, the final report, and a public experiment repository.

Success is assessed against comparisons registered in advance. The pass condition is a statistically significant reduction in irreversible-error rate at matched interruption frequency against B2 and B2+, with a 40 percent relative reduction stated as a target rather than a pass threshold, plus a significant reduction in interruptions per task at matched error rate, no loss of task success against the ungated agent, and frontier dominance under bootstrap testing.

6. Stakeholders

Enterprise teams in financial technology, customer support and internal automation cannot currently grant agents write access without accepting unbounded downside from a single wrong action, and are the primary users of the SDK and proxy. Compliance, risk and audit functions require a record of who approved which consequential action and when, and consume the audit log. Platform vendors benefit because repeated public incidents damage trust in the product category, and the proxy is designed for adoption without vendor cooperation. Human operators want to be consulted only where it matters, since over-interruption causes safeguards to be switched off or rubber-stamped, and customers bear the consequences of erroneous refunds and destroyed records. The research community lacks a labelled reversibility dataset and a shared taxonomy, both released openly here. The project guide, the Review Committee and the Department of CSE review the work at each phase boundary.

7. Gantt Chart

Period	Task	Deliverable
Aug to Sep 2026	Literature survey, problem formulation, title approval, synopsis	Approved synopsis
Sep to Oct 2026	Taxonomy design and inter-annotator validation	D1: taxonomy and schema
Oct to Nov 2026	Sandbox construction, rollback instrumentation, verifier tests	D2: sandbox
Nov to Dec 2026	Dataset generation, labelling, weight sensitivity study	D3: 10,000+ actions
Dec 2026 to Jan 2027	Regressor and conformal calibration; Paper I; mid-project review	D4: regressor; paper submitted
Jan to Feb 2027	Detector: contextual and compositional heads	D5: detector
Feb to Mar 2027	Policy engine: thresholds, budget, batching, staged screen	D6: policy engine
Feb to Apr 2027	Integration: proxy, SDK, approval console, audit log	D7: package and console
Mar to May 2027	Evaluation and ablations; Paper II, report, demonstration, viva	Frontier result; D8

Dataset generation cannot begin before the rollback verifier is tested, so slack is reserved in March and April 2027.

8. References

[1] Zhou, S., et al., "WebArena: A Realistic Web Environment for Autonomous Agents," ICLR, 2024.
[2] Yao, S., et al., "tau-bench: Tool-Agent-User Interaction Benchmark," 2024. arXiv:2406.12045.
[3] Jimenez, C.E., et al., "SWE-bench: Can LMs Resolve Real-World GitHub Issues?" ICLR, 2024.
[4] Ruan, Y., et al., "Identifying the Risks of LM Agents with an LM-Emulated Sandbox," ICLR, 2024.
[5] Debenedetti, E., et al., "AgentDojo: Prompt Injection Attacks and Defenses," NeurIPS, 2024.
[6] Greenblatt, R., et al., "AI Control: Improving Safety Despite Intentional Subversion," ICML, 2024.
[7] Ren, A.Z., et al., "Robots That Ask For Help: Uncertainty Alignment for LLM Planners," CoRL, 2023.
[8] Angelopoulos, A.N., Bates, S., "A Gentle Introduction to Conformal Prediction," 2021.
[9] Elmachtoub, A.N., Grigas, P., "Smart Predict, then Optimize," Management Science, 68(1), 2022.
[10] Garcia-Molina, H., Salem, K., "Sagas," ACM SIGMOD Conference on Management of Data, 1987.
[11] IBM Security and Ponemon Institute, "Cost of a Data Breach Report 2026," IBM, 2026.